`:delete` option to `link_to`.

Adding Additional Actions

Rails resources provide you with an initial set of actions, but you don't need to stop there. For example, if you want to add an interface to allow people to fetch a list of people who bought any given product, you can add an extension to the `resources` call:

```
Depot::Application.routes.draw do
  resources :products do
    get :who_bought, on: :member
  end
end
```

That syntax is straightforward. It says "We want to add a new action named `who_bought`, invoked via an HTTP `GET`. It applies to each member of the collection of products."

Instead of specifying `:member`, if we instead specified `:collection`, then the route would apply to the collection as a whole. This is often used for scoping; for example, you may have collections of products on clearance or products that have been discontinued.

Nested Resources

Often our resources themselves contain additional collections of resources. For example, we may want to allow folks to review our products. Each review would be a resource, and collections of reviews would be associated with each product resource. Rails provides a convenient and intuitive way of declaring the routes for this type of situation:

```
resources :products do
  resources :reviews
end
```

This defines the top-level set of product routes and additionally creates a set of subroutes for reviews. Because the review resources appear inside the products block, a review resource *must* be qualified by a product resource. This means